

eada business school barcelona

Why Convolutions? Filters as Features

SESSION 4.1

Clarifications After Session 4

1

What is a filter for?

We produced many filtered images. What do they give us?

2

How does a filtered image become a feature?

A filtered image is still 30,000 numbers. A model needs a few.

3

How many filters should we use?

Two? Ten? Sixty-four? How do we decide?

Three ideas to make clear, and then how a CNN does all of this automatically. Concepts only, no code.

Idea 1: A Filter Is a Question Asked at Every Pixel

Blur

1	1	1
1	1	1
1	1	1

× 1/9

“What is the average brightness around here?”

Sobel x

−1	0	1
−2	0	2
−1	0	1

“Is it brighter on the right than on the left?”

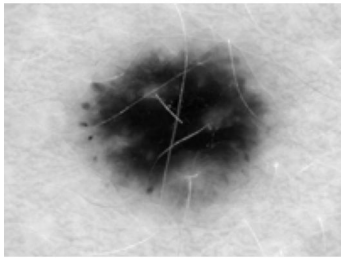
Laplacian

0	−1	0
−1	4	−1
0	−1	0

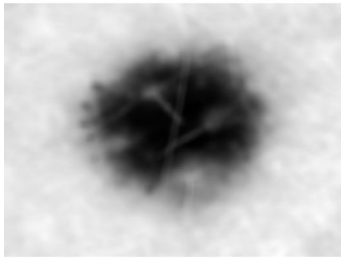
“Is this pixel different from its neighbors?”

- The 9 weights **are** the question. Positive weight: 'I want brightness here'. Negative: 'I want darkness here'.
- Convolution = asking the same question at every position of the image.

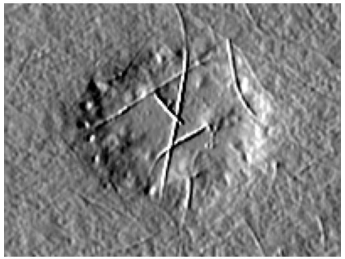
The Answer Is a New Image: a Feature Map



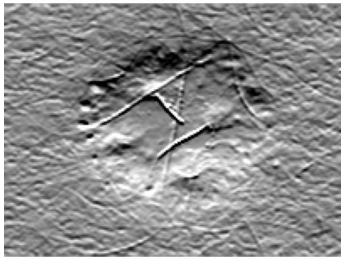
original



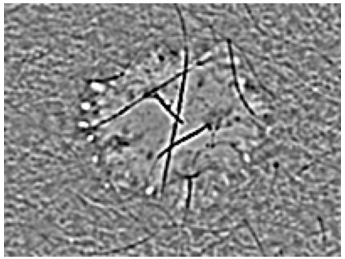
blur: average?



Sobel x: left ↔ right?



Sobel y: top ↕ bottom?



Laplacian: unlike neighbors?

- One answer per pixel, so the output is an image too: bright where the answer is 'yes'.
- Each filter makes **one kind of pattern** visible: shape (blur), vertical or horizontal borders (Sobel), spots and hairs (Laplacian).
- A filter does **not** decide anything. It only **highlights**. Deciding is the job of the model that comes after.

What Filters Were Used For in Classical Vision

1 — Clean

Remove noise

Gaussian blur, median

2 — Enhance

Make details clearer

sharpening

3 — Detect

Find edges and corners

Sobel; Canny, Harris**

4 — Describe

Turn texture and shape into numbers

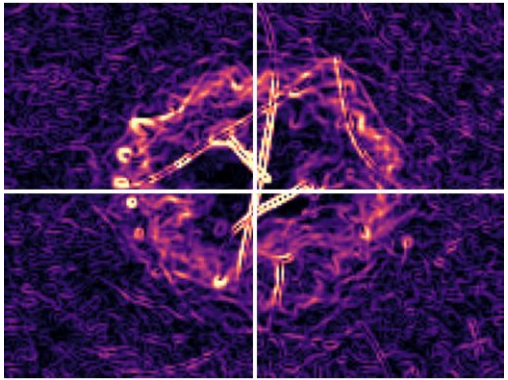
Gabor, LBP*, HOG*, SIFT**

- From the 1970s until about 2012, computer vision mostly worked like this: **people chose the filters**, then built features from their outputs.
- Step 4 is what we do in Session 5: filter outputs → numbers → a classifier.

* Not covered in class. A one-line summary and a reference for each are on the **Going Further** slide at the end.

Idea 2: From a Map to a Few Numbers

? *A feature map has 30,000 values and the lesion can be anywhere. How do we feed it to a model?*



Sobel edge strength, ISIC_0073863

summarize the map with…	value	it tells us
mean	25.9	how many edges overall
std	22.7	even or patchy edges
95th percentile	65.1	how strong the strongest edges are
fraction > 50	9.7%	how much area has edges
mean per quadrant	26.0 · 27.7 · 24.0 · 25.9	a little bit of WHERE

THE KEY STEP

Filter ⟶ feature map ⟶ **summarize** ⟶ a few numbers = features. CNNs also summarize maps; there it is called **pooling**.

Several Filters × Several Summaries = Feature Vector

	mean	std
blur	✓	✓
Sobel x	✓	✓
Sobel y	✓	✓
Laplacian	✓	✓
Gabor 45°	✓	✓

5 filters, 10 ticks ⟶ **10 numbers** per image

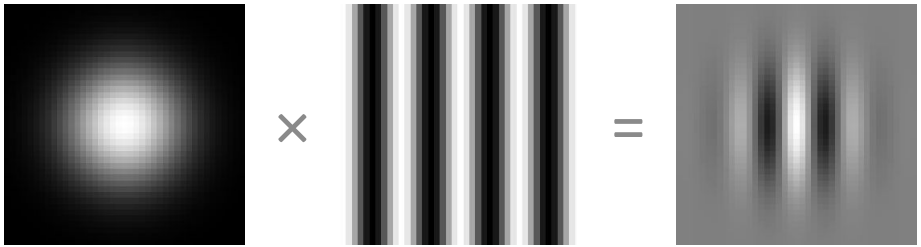
- **Rows** = filters: what we look for.
- **Columns** = summaries: how we count it.
- Each tick is one feature. Put them in a row ⟶ one vector per image ⟶ any classifier.

SO, A FEATURE IS…

…one summary number of one filter's map. Filters choose **what** to look for; summaries choose **how** to count it.

SESSION 4.1

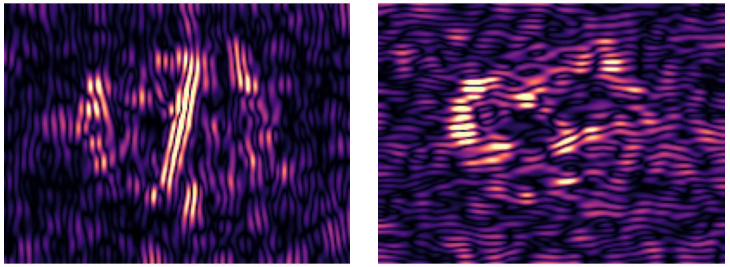
What Is a Gabor Filter?



a soft round window | a stripe pattern | **Gabor filter**

- A Gabor filter is just a **small patch of stripes**, faded out at the edges.
- Its question: *'are there stripes or edges in THIS direction, with THIS thickness, here?'*
- Three knobs: **direction** (angle), **stripe width**, **window size**.
- Sobel = one fixed direction and size. Gabor = any direction, any size: ideal for **texture**.

SAME LESION, TWO GABOR FILTERS



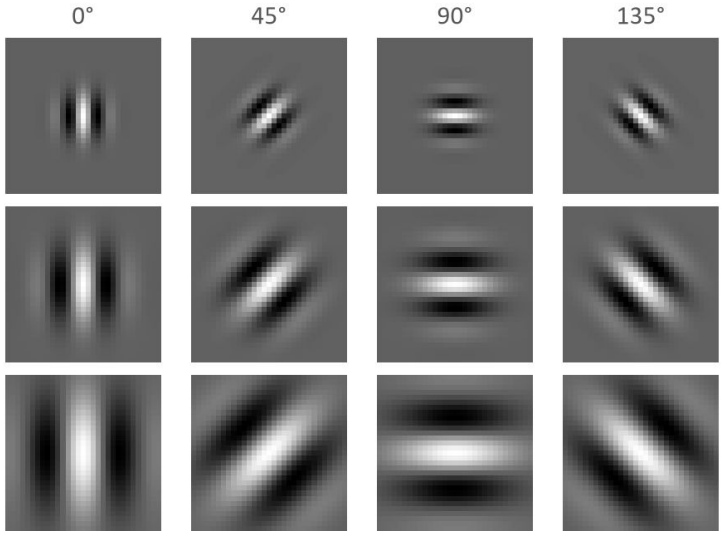
vertical stripes (0°) | horizontal stripes (90°)

Each one lights up hairs and borders running in its own direction.

Fun fact: neurons in our visual cortex respond to stripes in a similar way, and CNNs learn Gabor-like filters on their own.

Idea 3: How Many Filters? No Magic Number

- **Ask different questions**, not the same one many times: something smooth, edges in a few directions, spots.
- **Look at different sizes**: hairs are thin, globules medium, the lesion border large.
- **Typical numbers**: Sobel = 2 filters; a Gabor bank = 4 directions × 3 sizes = 12.
- **More is not always better**: similar filters give similar features, and with few images the model overfits.
- **Decide with data**: add a group of filters, check the validation score, keep it only if it helps.



A Gabor bank: 4 directions (columns) × 3 sizes (rows) = 12 filters

Rule of thumb: start small (3–12 filters), measure, then grow.

The Limit: We Have to Choose the Questions

! Hand-picked

We only find what we thought of looking for. Who can design a filter for a 'pigment network'?

! No context

An edge filter can't tell a hair from a lesion border: both are strong edges.

! No combinations

'An edge here AND a dark spot next to it' needs filters applied on top of filters.

THE IDEA BEHIND CNNs

What if the computer chose the numbers inside each filter by itself, learning from thousands of labelled images, and stacked filters on top of filters?

A CNN: Same Filters, but Learned

BY HAND Session 4

−1	0	1
−2	0	2
−1	0	1

we choose the 9 numbers



CNN Session 7

?	?	?
?	?	?
?	?	?

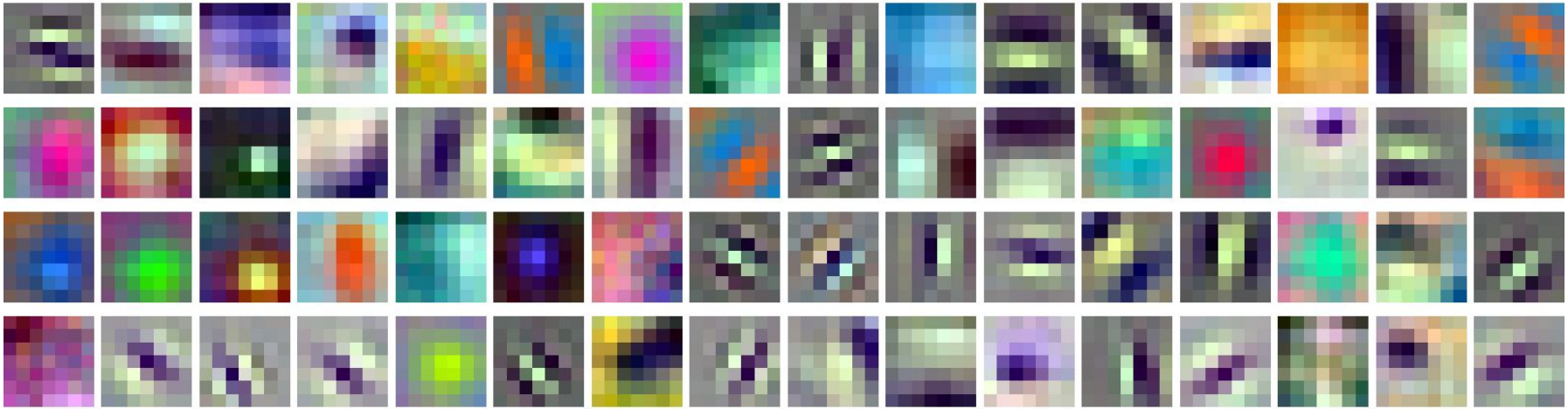
the data chooses them

1. Start with **random** numbers
2. Make predictions on training images
3. Measure how wrong they are
4. Adjust every number a little to be less wrong
5. Repeat thousands of times

- The operation is **exactly** the one from Session 4: slide, multiply, sum. Only the origin of the numbers changes.
- If an edge detector is useful for the task, training finds one by itself. If not, it finds something else.

SESSION 4.1

What CNNs Actually Learn



The 64 first-layer filters of a real CNN (ResNet-18), learned from 1.2 million photos. Nobody designed them.

Layer 1
edges, stripes, colors
≈ Sobel, Gabor

→

Layers 2–3
textures, corners
≈ our texture features

→

Deeper
parts, structures
e.g. globules, networks

→

Last
the class
Benign / Malignant

SESSION 4.1

How Many Filters in a CNN?

- A **design choice**: usually 32 or 64 in the first layer, **doubling** as we go deeper.
- We give the network **more filters than needed**; training decides what each one looks for.
- Deeper filters look at the **maps of the previous layer**, so they combine simple patterns into complex ones.
- **Pooling** summarizes the maps, like our mean and max.

ResNet-18	filters	numbers to learn
layer 1	64	9,408
stage 2	64	≈150 k
stage 3	128	≈525 k
stage 4	256	≈2.1 M
stage 5	512	≈8.4 M

COMPARE

Session 5: **we** chose N texture filters. ResNet-18: about **11 million** learned numbers.
That's why CNNs need much more data and computing power.

Classical Pipeline vs. CNN, Side by Side

	Classical (Sessions 4–5)	CNN (Session 7 onwards)
Filters	Chosen by us	Learned from labelled images
How many	A few, tested on validation	Dozens to hundreds per layer
Summarize	mean, std, histogram…	Pooling
Decide	A separate classifier (e.g. forest)	The last layers, trained together
Needs	Little data, a laptop	Lots of data, a GPU
Strength	Fast, easy to explain	Finds patterns we never thought of

*Same building block in both columns: convolution. What changes is **who** chooses the filters.*

Key Takeaways

1. **What for.** A filter asks one question at every pixel and returns a map that highlights one pattern.

2. **Features.** Summarize each map (mean, std, …): several filters × several summaries = feature vector.

3. **How many.** No magic number: different questions and sizes, start small, keep what improves validation.

4. **CNNs.** Same convolution, but the filters are learned and stacked; the first layer learns Sobel- and Gabor-like filters.

SESSION 4.1 · OPTIONAL

Going Further: Classical Methods Not Covered in Class

Not needed for the course. The idea of each in one line, where to try it, and the original paper.

Method	What it does	Try it	Reference
Canny	Clean, thin edge lines: blur, Sobel, keep only the sharpest edge pixels, link them.	cv2.Canny	Canny, IEEE TPAMI, 1986
Harris	Finds corners: spots where brightness changes in two directions at once.	cv2.cornerHarris	Harris & Stephens, Alvey Vision Conf., 1988
Gabor	Stripe detectors of a chosen direction and size; a bank of them describes texture.	skimage.filters.gabor	Daugman, J. Opt. Soc. Am. A, 1985
LBP	Compares each pixel with its 8 neighbors → an 8-bit code; the histogram of codes = texture.	skimage.feature. local_binary_pattern	Ojala et al., IEEE TPAMI, 2002
HOG	Splits the image into cells; per cell, a histogram of edge directions = shape.	skimage.feature.hog	Dalal & Triggs, CVPR, 2005
SIFT	Finds distinctive keypoints at several sizes and describes them, to match objects across photos.	cv2.SIFT_create	Lowe, Int. J. Comput. Vision, 2004